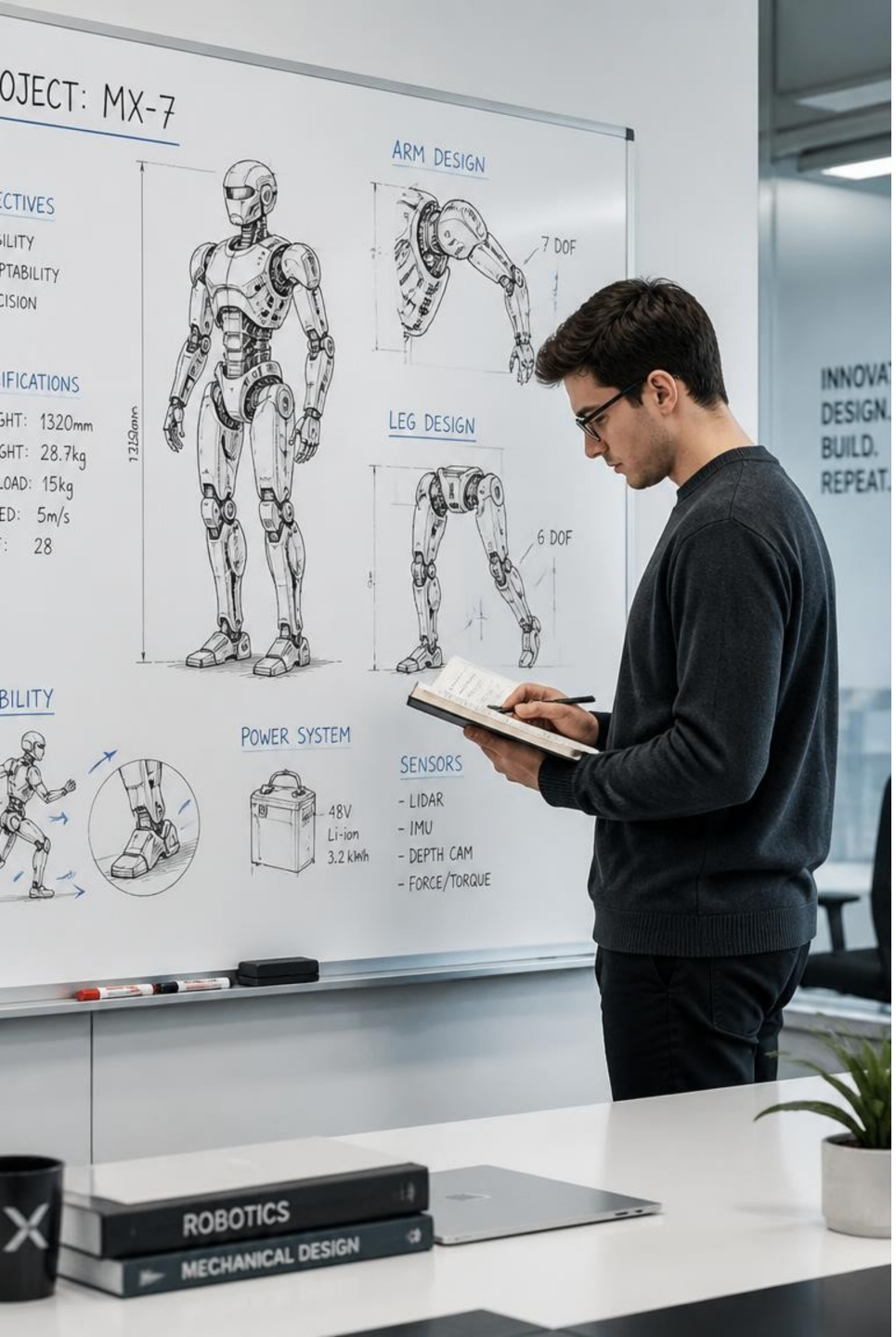


روبوت المحادثة والذكاء الاصطناعي — الجزء الثاني

اسم المقرر: أنظمة تشغيل الروبوتات | **الأسبوع الثاني عشر** | مدة الشريحة: 2 ساعة

د. سلطان النفيعي - 2026



مراجعة سريعة — ماذا تعلمنا الأسبوع الماضي؟

01

الروبوتات الاجتماعية

مقدمة في الروبوتات التي تتفاعل مع البشر

02

لغة AIML للمحادثة

بنية اللغة وطريقة كتابة القواعد

03

Template g Pattern

العنصران الأساسيان في كل قاعدة AIML

04

مكتبة PyAIML و Kernel

تشغيل المحادثة في Python وإدارة محرك AIML

ما سنتعلمه اليوم

بناء روبوت محادثة في ROS

تصميم النظام الكامل من الصفر

حزم ROS للصوت

إعداد المكتبات والأدوات الصوتية

تكامل AIML مع ROS

ربط محرك المحادثة بعقد ROS

STT و TTS

تحويل الكلام لنص والنص لكلام

هيكل المشروع — Project Structure

chatbot_aiml/

حزمة قواعد AIML — قلب منطق المحادثة

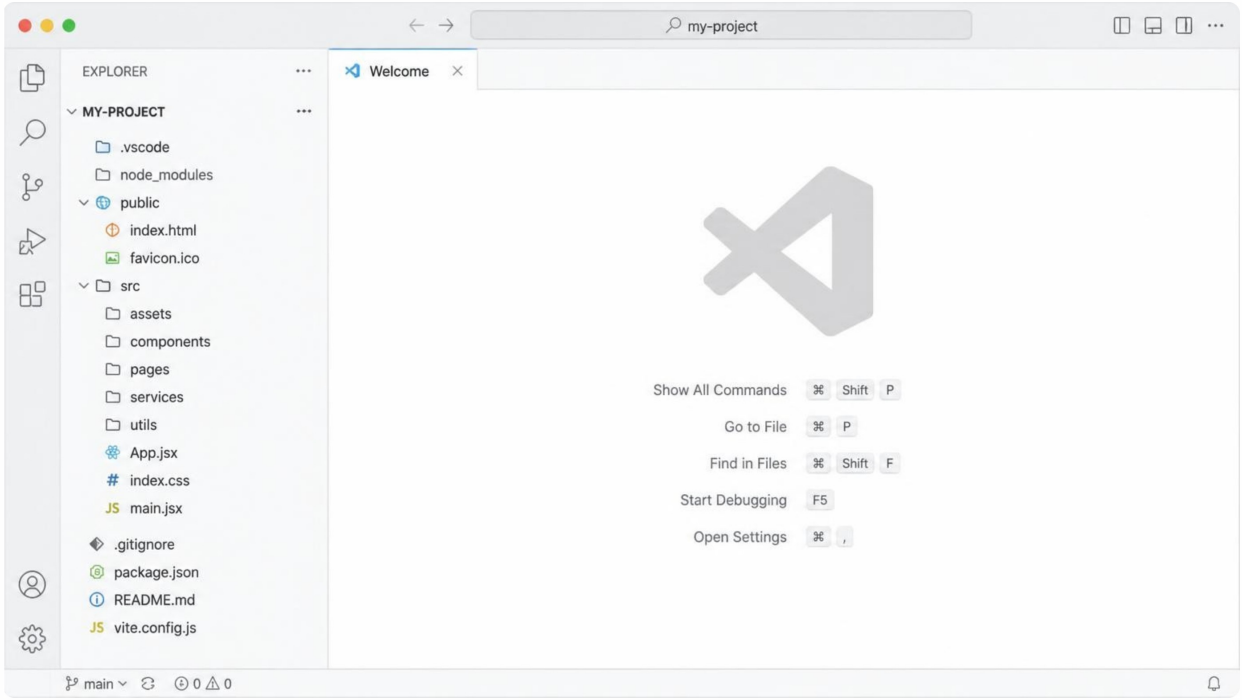
chatbot_audio/

حزمة الصوت — الاستماع والتحدث

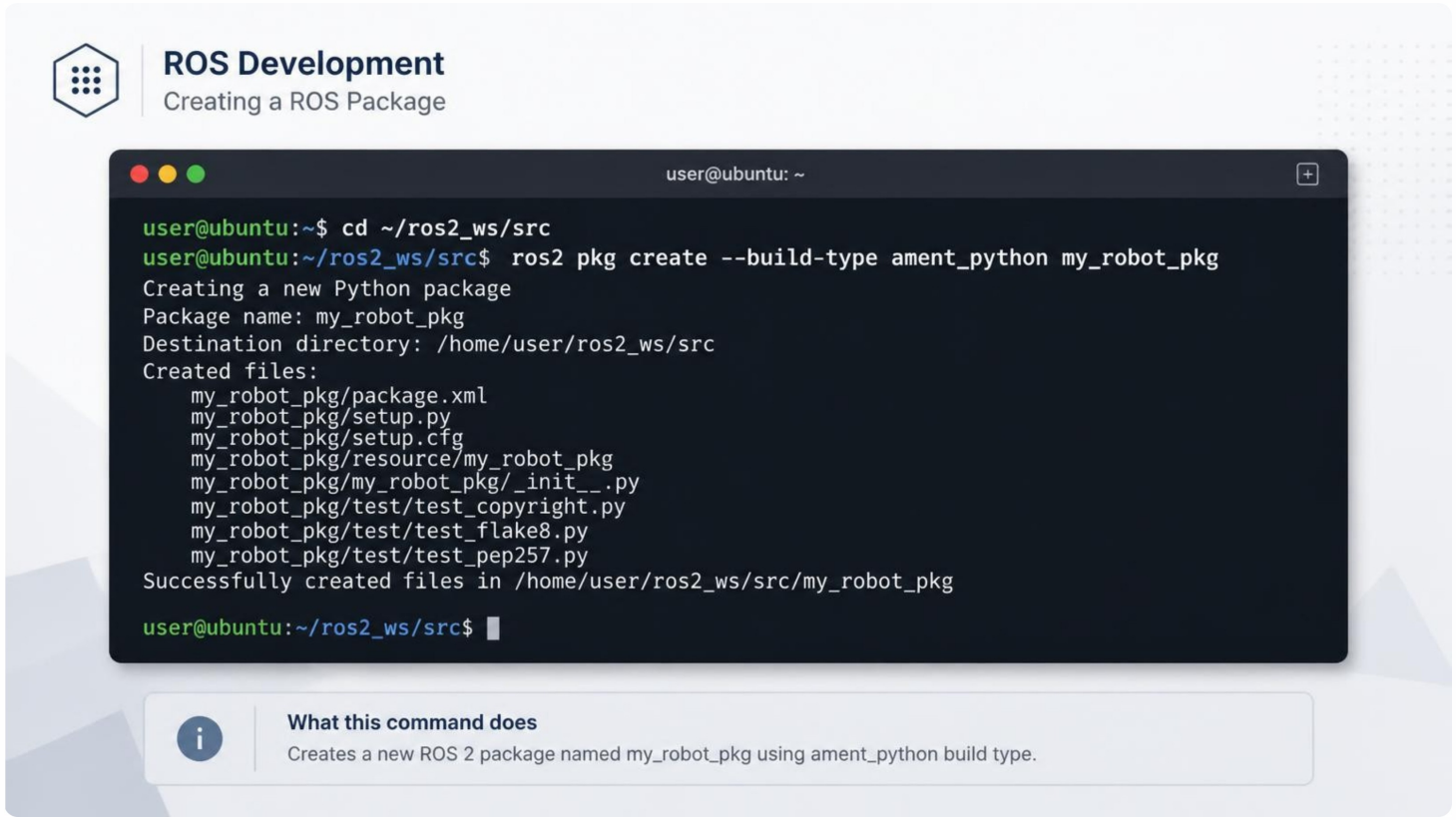
chatbot_nodes/

عقد ROS — تنسيق تدفق البيانات

كل حزمة مستقلة وقابلة للتطوير بشكل منفصل



إنشاء حزمة AIML



الخطوات في الطرفية

انتقل أولاً إلى مجلد المصدر:

```
cd ~/catkin_ws/src
```

ثم أنشئ الحزمة بالأمر:

```
catkin_create_pkg chatbot_aiml
                  rospy std_msgs
```

هذا ينشئ حزمة جديدة تعتمد على **rospy** و **std_msgs** وتخصص لملفات AIML فقط.

هيكل حزمة AIML



aiml/questions.aiml

قواعد الأسئلة الشائعة والإجابات



aiml/greetings.aiml

قواعد التحيات والترحيب بالمستخدمين



launch/load_aiml.launch

ملف التشغيل لتحميل جميع قواعد AIML



aiml/standard.aiml

قواعد عامة لتغطية المواضيع المختلفة

ملف AIML للتحيات — greetings.aiml

Pattern (المدخل)	Template (الرد)
مرحبا	أهلاً وسهلاً بك!
السلام عليكم	وعليكم السلام ورحمة الله
صباح الخير	صباح النور!
وداعاً	مع السلامة!

ملف AIML للأسئلة — questions.aiml

Pattern (السؤال)	Template (الإجابة)
ما اسمك	اسمي روبوت ROS
كم عمرك	عمري سنة واحدة
من صنعك	صنعني طلاب تقنية الروبوتات
ماذا تفعل	أتحدث مع البشر وأساعدهم

تثبيت مكتبات الصوت

نفّذ الأوامر التالية بالترتيب في الطرفية:

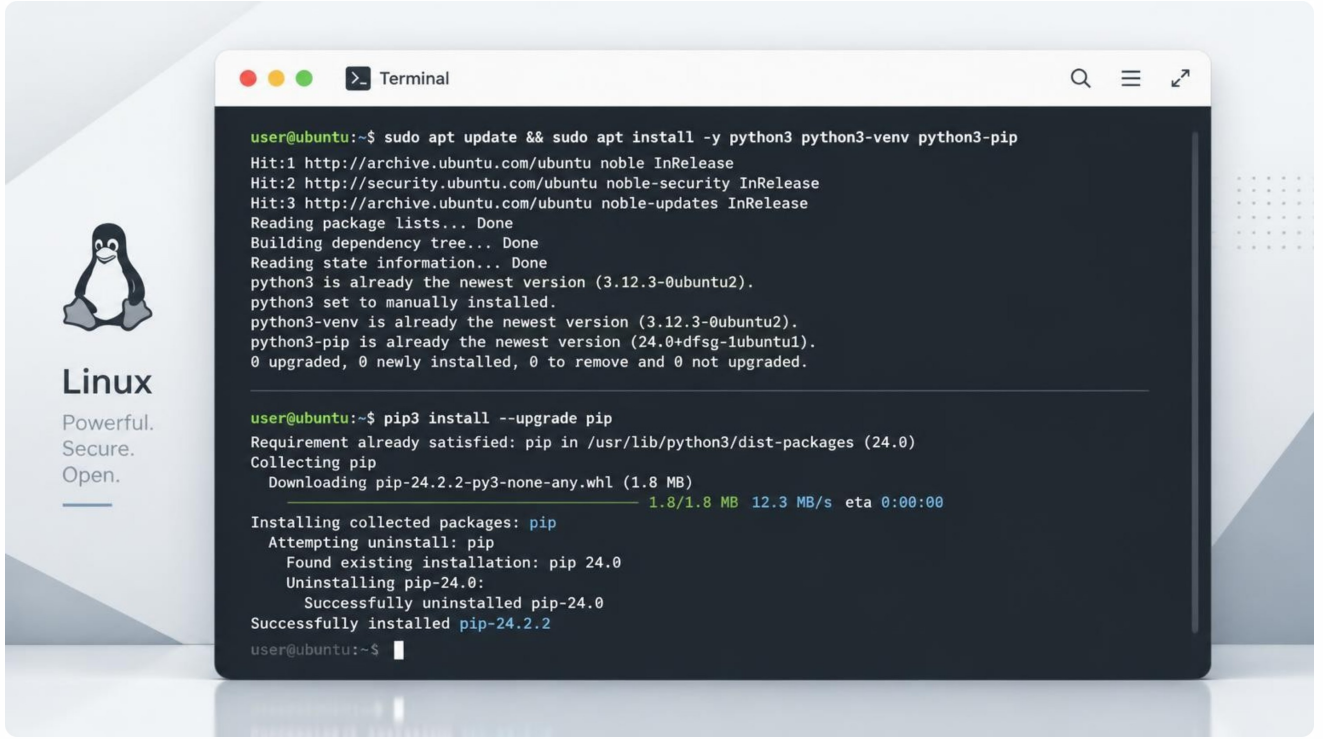
```
sudo apt install ros-noetic-sound-play
```

```
sudo apt install festival
```

```
pip3 install SpeechRecognition
```

```
pip3 install gTTS
```

تأكد من اتصال الإنترنت قبل التثبيت



تحويل النص لكلام — TTS



ما هو TTS؟

TTS = Text-to-Speech

تقنية تحوّل النص المكتوب إلى كلام مسموع، مما يتيح للروبوت التواصل صوتياً مع المستخدمين.

gTTS

Google TTS مع دعم كامل للعربية

sound_play

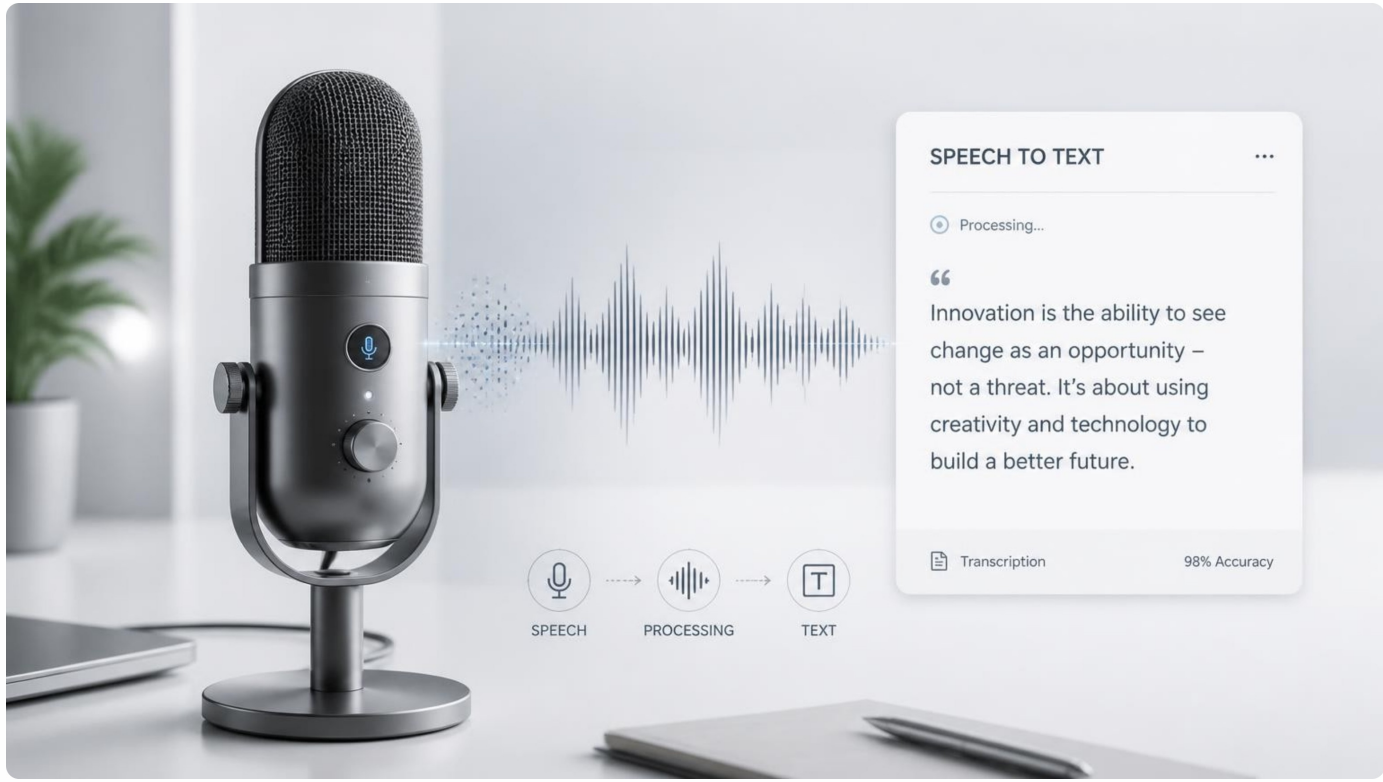
مكتبة ROS الرسمية للصوت

تحويل الكلام لنص — STT

ما هو STT؟

STT = Speech-to-Text

تقنية تحوّل الكلام المسموع إلى نص مكتوب ليتمكن الروبوت من فهم ما يقوله المستخدم.



مكتبة SpeechRecognition

الأداة الرئيسية للتعرف على الكلام في Python

دعم العربية

باستخدام المعامل: **'language='ar-SA**



عقدة الاستماع — Listener Node

الاستماع المستمر

تراقب الميكروفون باستمرار وتنتظر كلام المستخدم

التحويل عبر Google Speech API

تحول الكلام المسموع إلى نص مكتوب بدقة عالية

النشر على /user_speech

تنشر النص المحوّل على الموضوع لبقية العقد — بمعدل مرة كل ثانيتين

عقدة المعالجة — Processor Node

دور عقدة المعالجة

- الاشتراك في موضوع **user_speech/** لاستقبال كلام المستخدم
- إرسال النص إلى **AIML Kernel** لمعالجته
- استقبال الرد الجاهز من Kernel
- نشر الرد على موضوع **bot_response/**





عقدة التحدث — Speaker Node

1

استقبال الرد

الاشتراك في موضوع **bot_response/**

2

التحويل لصوت

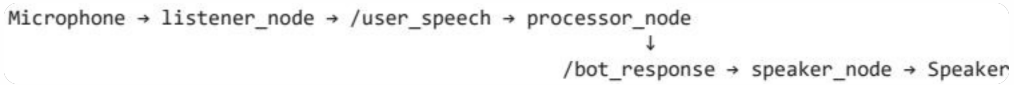
تحويل النص لكلام باستخدام TTS

3

النشر الصوتي

النشر على **robotsound/** ليسمعه المستخدم

تدفق البيانات — Data Flow



المايكروفون ← listener_node
/user_speech ← processor_node
/bot_response ← speaker_node ← المكبر

مواضيع النظام — Topics

/robotsound

الصوت النهائي المُشغَّل للمستخدم

النوع: **SoundRequest**

/bot_response

رد الروبوت الصادر من AIML

النوع: **String**

/user_speech

كلام المستخدم المحوّل لنص

النوع: **String**

ملف Launch الرئيسي — chatbot.launch

ملف واحد يشغّل النظام بالكامل دفعة واحدة:

1. تحميل AIML

عقدة تقرأ جميع ملفات AIML وتهيئ الـ Kernel

2. عقدة الاستماع (listener)

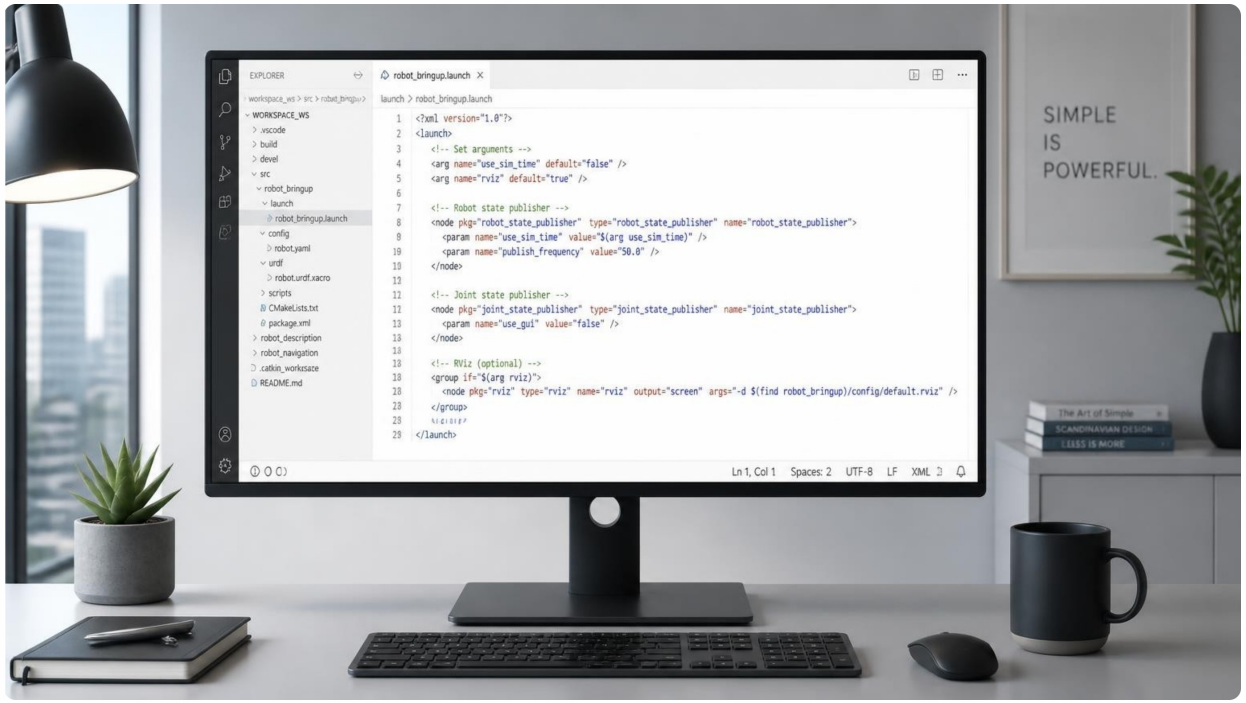
تبدأ مراقبة الميكروفون

3. عقدة المعالجة (processor)

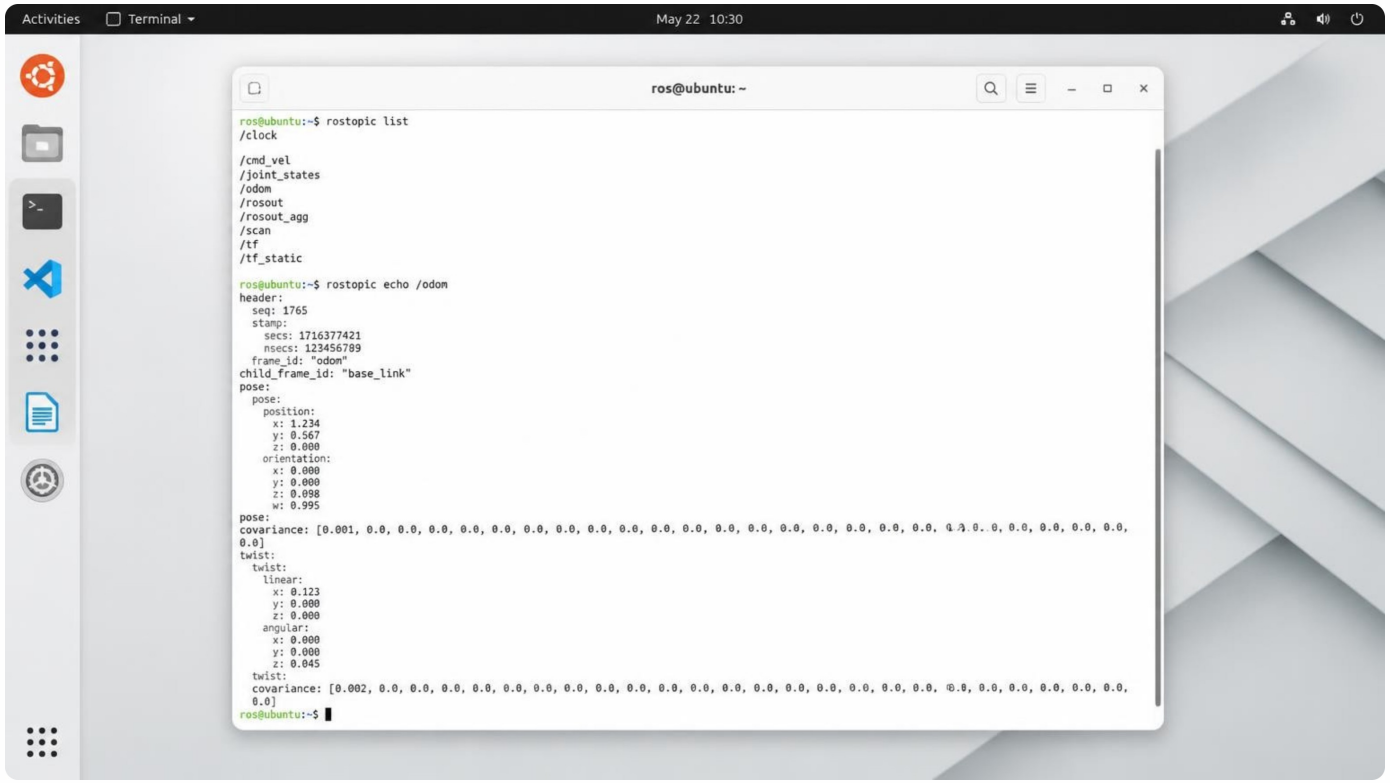
تربط الكلام بمحرك AIML

4. عقدة التحدث (speaker)

تحوّل الردود لصوت مسموع



تشغيل النظام



أمر التشغيل الرئيسي

```
roslaunch chatbot_audio
chatbot.launch
```

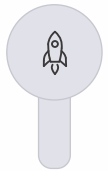
اختبار بدون ميكروفون

```
rostopic pub /user_speech
"'╔╠ᴰᵒ_' :std_msgs/String "data
```

مراقبة الرد

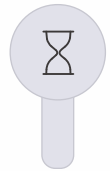
```
rostopic echo /bot_response
```

اختبار النظام — الخطوات



شغّل ملف launch

ابدأ النظام بأمر roslaunch



انتظر تحميل العقد

تأكد من تشغيل جميع العقد بنجاح



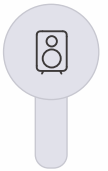
قل "مرحبا" بوضوح

تحدث باتجاه الميكروفون بصوت واضح



تحقق من /user_speech

راقب الموضوع للتأكد من استقبال النص



استمع لرد الروبوت

يصدر الرد صوتياً عبر المكبر

مشكلة شائعة — Audio Issues



أدوات تشخيص الصوت

تحقق من الميكروفون المتاح:

```
arecord -l
```

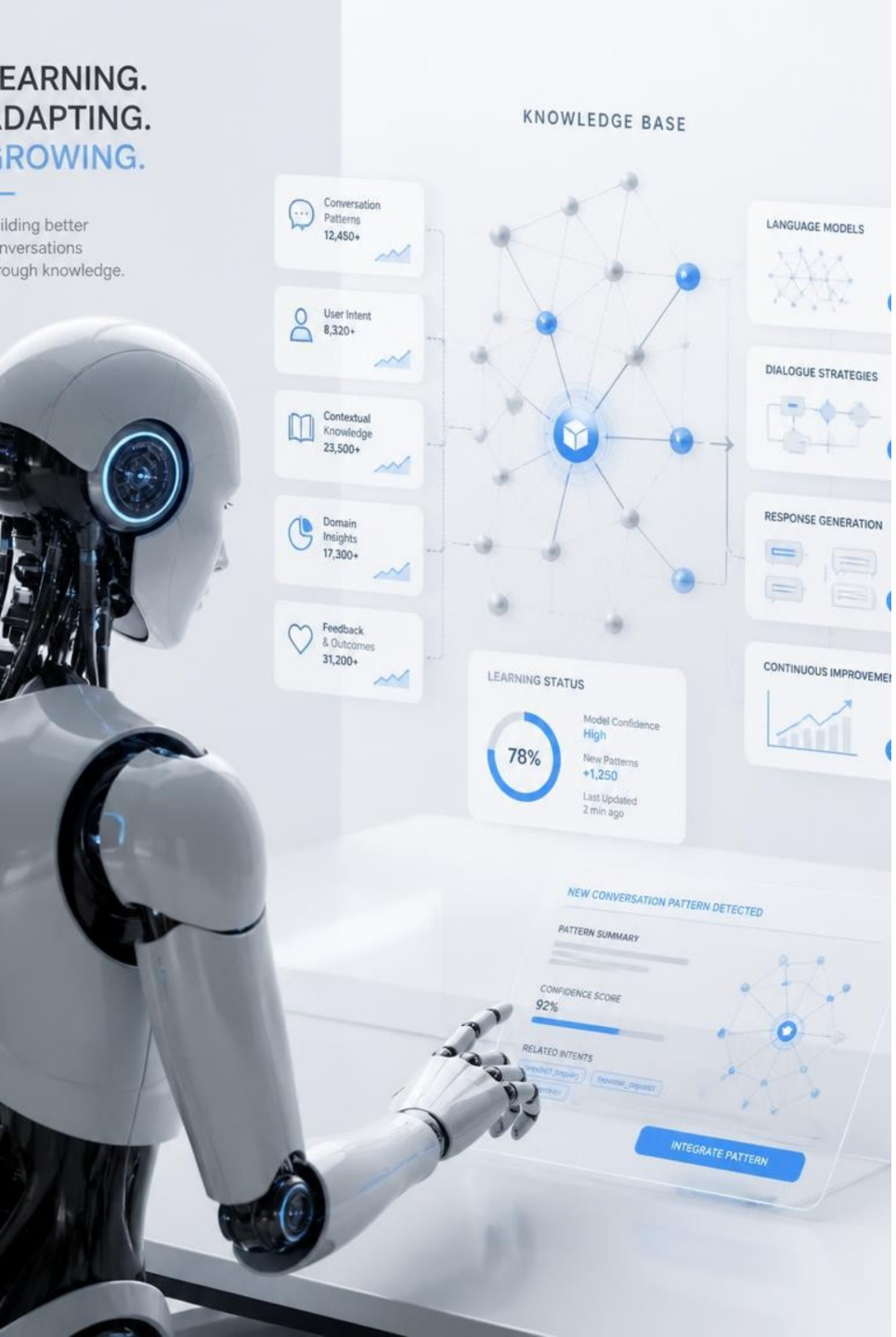
تحقق من المكبر المتاح:

```
aplay -l
```

اختبر الميكروفون مباشرة:

```
arecord test.wav && aplay test.wav
```

⚠ مستخدمو WSL: قد يحتاج الصوت إعداداً إضافياً — راجع توثيق PulseAudio لـ WSL



تحسين الردود — إضافة قواعد

قاعدة الطقس

Pattern: **"* طقس *"** → "لا أعرف الطقس، أنا روبوت داخلي"

قاعدة الوقت

Pattern: **"* وقت *"** → "الوقت يمر بسرعة!"

ردود عشوائية

إضافة ردود متعددة لنفس القاعدة لمحادثة أكثر طبيعية وتنوعاً

توسيع قاعدة البيانات

كلما زادت القواعد كانت المحادثة أفضل وأكثر ذكاءً

حفظ Brain — تسريع التحميل



حفظ الـ Brain بعد التحميل

```
kernel.saveBrain("bot.brn")
```

تحميل سريع في المرة القادمة

```
kernel.loadBrain("bot.brn")
```

ملف .aiml	بطيء — يُحلّل في كل تشغيل
ملف .brn	سريع جداً — جاهز للتحميل الفوري

تكامل AIML مع ROS — Class كاملة

بناء Class موحدة

نجمع جميع مكونات النظام في كلاس Python واحدة منظمة:

01

تهيئة في __init__

إنشاء العقدة وتهيئة AIML Kernel وتحميل الملفات

02

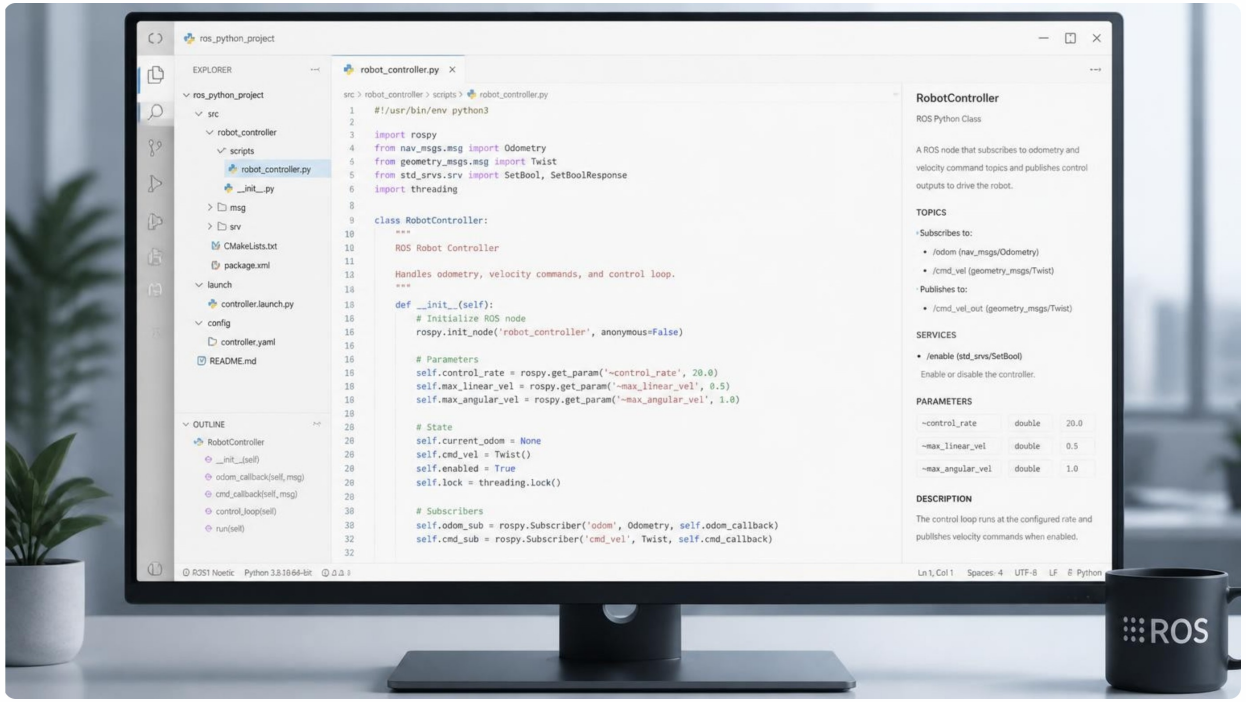
الاشتراك في user_speech/

استقبال كلام المستخدم كنص مكتوب

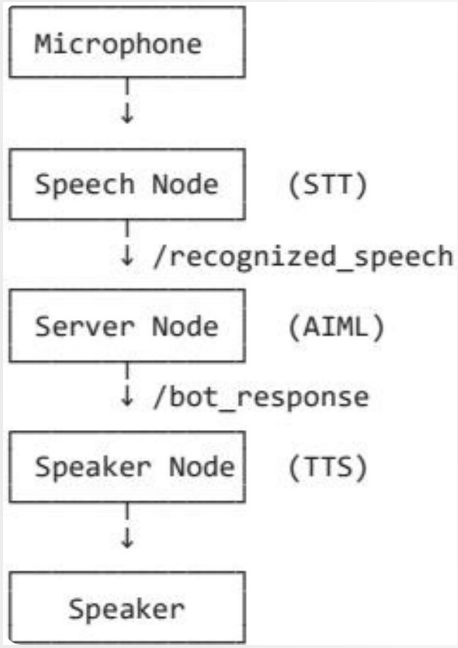
03

النشر على bot_response/

إرسال الرد لبقية العقد



رسم بياني للنظام



1

Speech Node (STT)

Microphone → /recognized_speech

2

Server Node (AIML)

/recognized_speech → /bot_response

3

Speaker Node (TTS)

/bot_response → Speaker

نصائح لتحسين الأداء

1

استخدم ملف .brn

بدلاً من تحميل AIML مباشرة لتوفير وقت الإقلاع

2

شغّل العقد الضرورية فقط

لا تشغّل عقداً غير مستخدمة لتقليل استهلاك الذاكرة

3

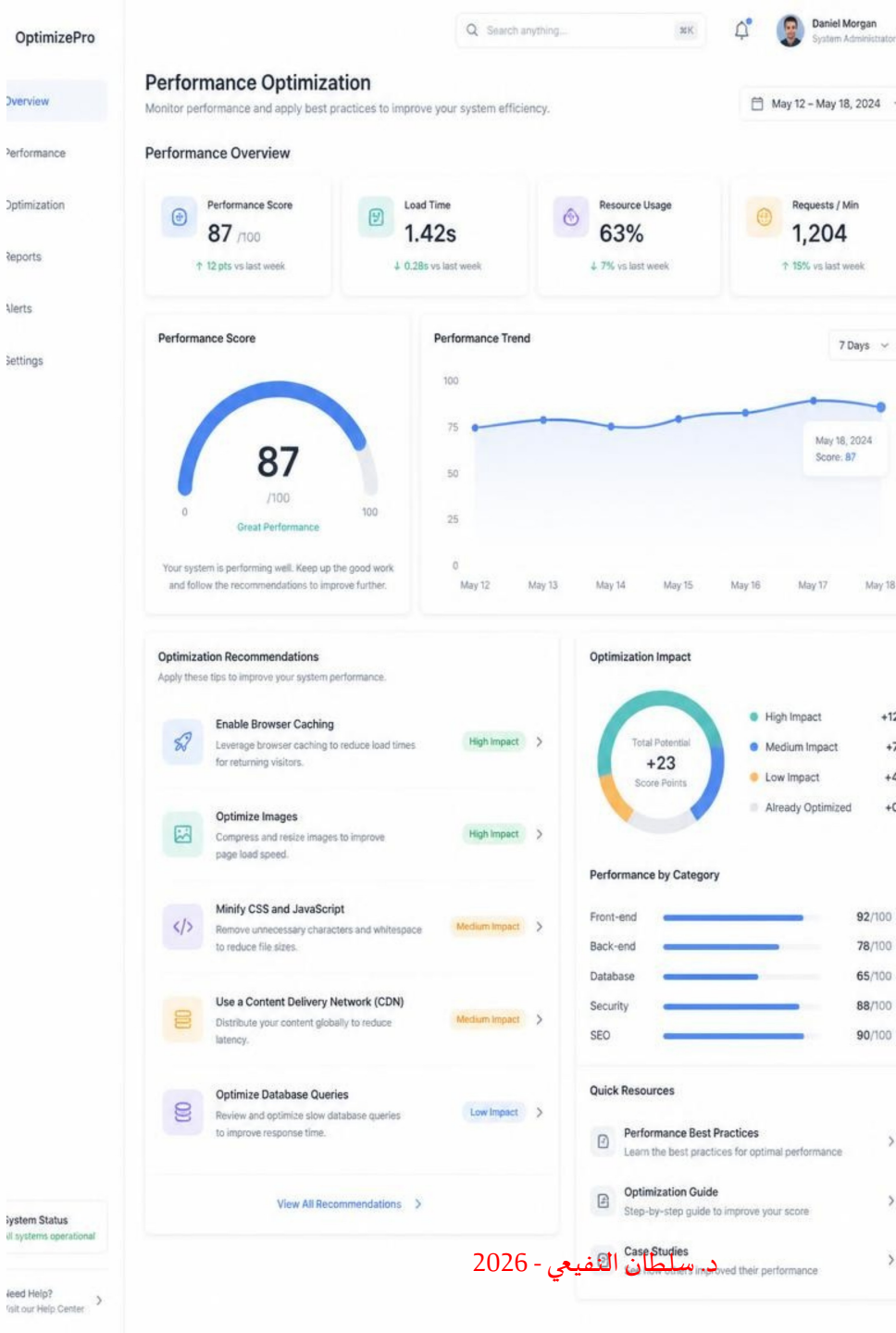
قلل معدل الاستماع

تقليل تكرار الاستماع يوفر موارد المعالج

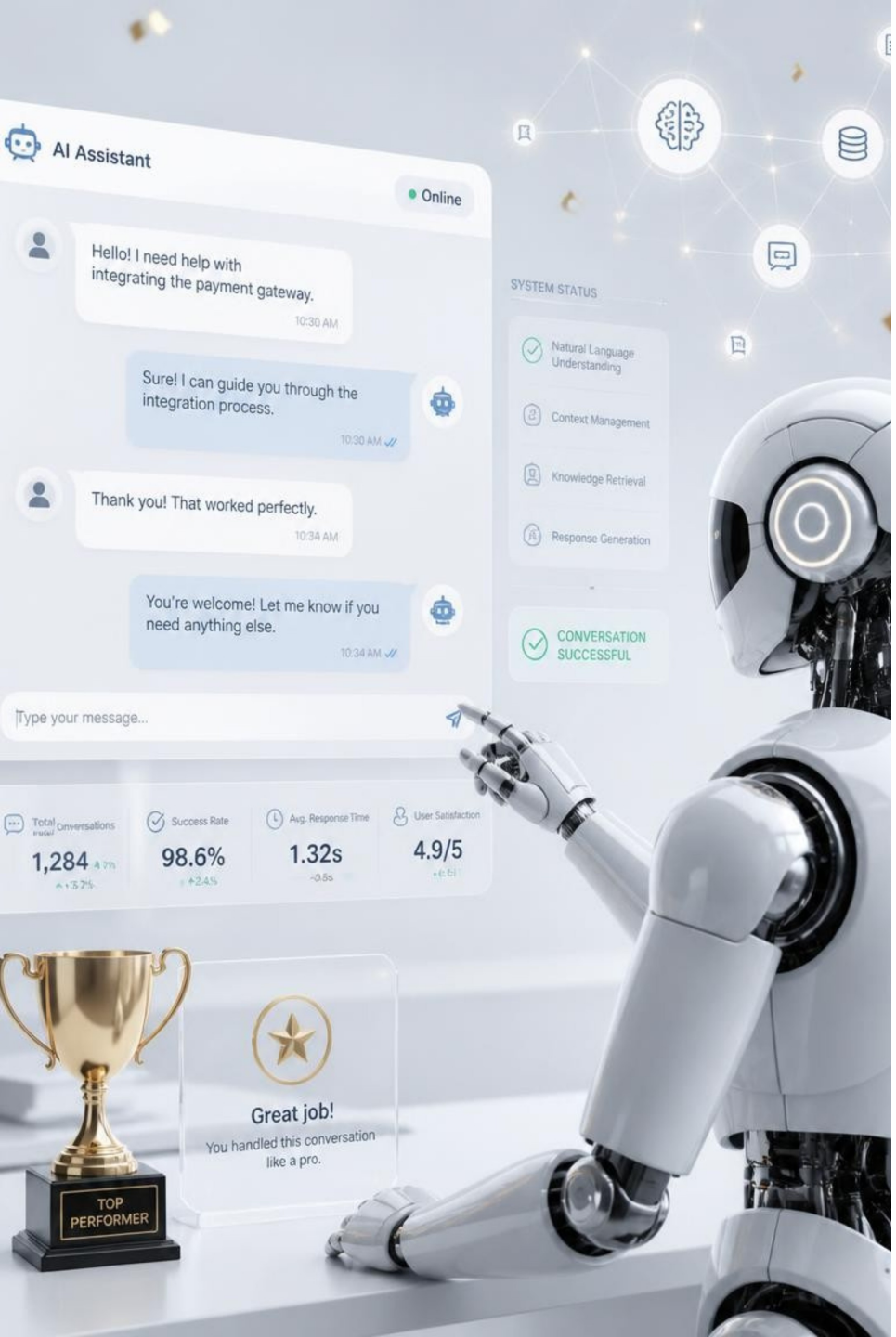
4

استخدم VAD

Voice Activity Detection لتجاهل الضوضاء الخلفية



د. سلطان الفيفي - 2026



ملخص الدرس — ما تعلمناه اليوم

✓ حزم ROS

بناء حزم للـ AIML والصوت

✓ العقد الثلاث

الاستماع، المعالجة، والتحدث

✓ تكامل AIML مع ROS

ربط محرك المحادثة بالنظام

✓ STT و TTS

نظام محادثة صوتي متكامل

أسئلة للمراجعة — اختبر نفسك

1 ما وظيفة عقدة listener؟

فكّر في دورها في سلسلة تدفق البيانات

2 ما الفرق بين STT و TTS؟

الاتجاه، الأدوات، وموضع كل منهما في النظام

3 ما المواضيع المستخدمة في نظام المحادثة؟

اذكر الثلاثة مع نوع البيانات لكل منها

4 لماذا نستخدم .brn بدلاً من .aiml؟

فكّر في الأداء وسرعة التحميل

5 كيف تختبر النظام بدون ميكروفون؟

ما الأمر الذي يحاكي إدخال الصوت؟



الأسبوع القادم — ماذا سنتعلم؟

Client Node g Server Node

بناء خدمات ROS للمحادثة المتقدمة

تحليل الصوت المتقدم

تقنيات أعمق لمعالجة الإشارات الصوتية

التعرف على الكلام بدقة أعلى

نماذج متطورة للتعرف على اللغة العربية

تشغيل النظام الكامل

تجميع كل المكونات في نظام موحد ومتكامل

📌 واجب: أنشئ 20 قاعدة AIML جديدة وجرب النظام على جهازك